PROJECT REPORT

Fake News Detection Using Natural Language Processing (NLP)

Submitted as part of

AI/ML with Python Training Program – September 2026 Batch

Student Name	Amruta Dipak Gaikwad
Roll Number	AIML-A6/SEP-11774

Academic / Training Project Submission

1. Student Details

Student: Amruta Dipak Gaikwad
Roll number: AIML-A6/SEP-11774
Program: AI/ML with Python Training Program – September 2026 Batch
Project title: Fake News Detection Using Natural Language Processing (NLP)

2. Abstract

This project develops a machine-learning application that classifies English-language news article text as Fake or Real using Natural Language Processing. The ISOT Fake News dataset is cleaned and explored, text is represented using Term Frequency–Inverse Document Frequency (TF-IDF), and two classifiers—Logistic Regression and Multinomial Naive Bayes—are trained and evaluated. The trained vectorizer and models are saved with Joblib and loaded by a Streamlit application for interactive predictions. On the held-out test set, Logistic Regression achieved an accuracy of 98.98% and an F1-score of 98.89%. These results describe performance on this dataset and should not be interpreted as proof that the system can verify claims or detect all misinformation in real-world news.

3. Introduction and Problem Statement

Online news is published and shared rapidly. Alongside reliable reporting, misleading or fabricated content can circulate and influence readers. Manually checking every article is time-consuming. Text classification can help identify patterns associated with examples labelled fake or real in a dataset.

Problem statement: Build a reproducible NLP classification pipeline that accepts news text and predicts whether its writing resembles the Fake or Real examples in the selected training dataset. The output is a model prediction, not an independent fact-check.

4. Objectives and Scope

• Inspect and understand the ISOT Fake News dataset.

• Clean and normalize article headlines and body text consistently.

• Convert text into numerical TF-IDF features without fitting on the test set.

• Train and compare Logistic Regression and Multinomial Naive Bayes.

• Evaluate using accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices.

• Save trained artifacts and build a Streamlit interface for interactive predictions.

• Present dataset and evaluation visualizations in an application dashboard.

The scope is limited to English text classification using the available dataset and two traditional machine-learning algorithms. The application does not retrieve external evidence, verify publishers, or establish the factual truth of an article.

5. Dataset Source and Description

The project uses the ISOT Fake News dataset provided by the Information Security and Object Technology (ISOT) Research Lab at the University of Victoria. The dataset contains two CSV files: Fake.csv and True.csv. The official dataset description states that truthful articles were collected from Reuters, while fake-news examples were collected from sources identified as unreliable by Politifact and other references. The dataset contains article title, text, subject/type and publication date.

Official source: https://onlineacademiccommunity.uvic.ca/isot/2022/11/27/fake-news-detection-datasets/
Dataset documentation:
https://onlineacademiccommunity.uvic.ca/isot/wp-content/uploads/sites/7295/2023/02/ISOT_Fake_News_Dataset_ReadMe.pdf

The original CSV files inspected by the EDA script contain 44,898 rows: 23,481 Fake and 21,417 Real. The project does not redistribute the dataset in this report. Users should obtain it from the original source and follow the source's applicable terms and attribution requirements; this report does not assert a separate licence.

Dataset inspection output generated by the project:

Column	Data Type	Missing Values
title	str	0
text	str	0
subject	str	0
date	str	0
label	str	0
article_length	int64	0

6. Data Preprocessing

The training pipeline combines the article title and body text. The preprocessing function in src/preprocess.py normalizes the text by lowercasing it and removing unwanted elements such as URLs, HTML fragments, digits and punctuation. The Reuters source tag is removed to reduce the risk of the classifier learning a direct source marker instead of broader text patterns. Empty cleaned records are excluded and exact duplicate cleaned texts are removed before splitting.

The same clean_text function is used by the Streamlit application before transforming user input. This keeps the training and prediction preprocessing steps consistent.

7. Exploratory Data Analysis (EDA)

The EDA script (python -m src.eda) inspects the raw CSV dataset, including its dimensions, column names, data types, missing values and class distribution. It also calculates article word lengths and produces plots for label distribution, article length and frequent words in Fake and Real articles.

The raw EDA count (44,898) differs from the model dataset count because the training pipeline cleans text, removes empty records and removes duplicate cleaned articles. After those operations, 38,824 articles are used for model development. The counts describe different processing stages and should not be conflated.

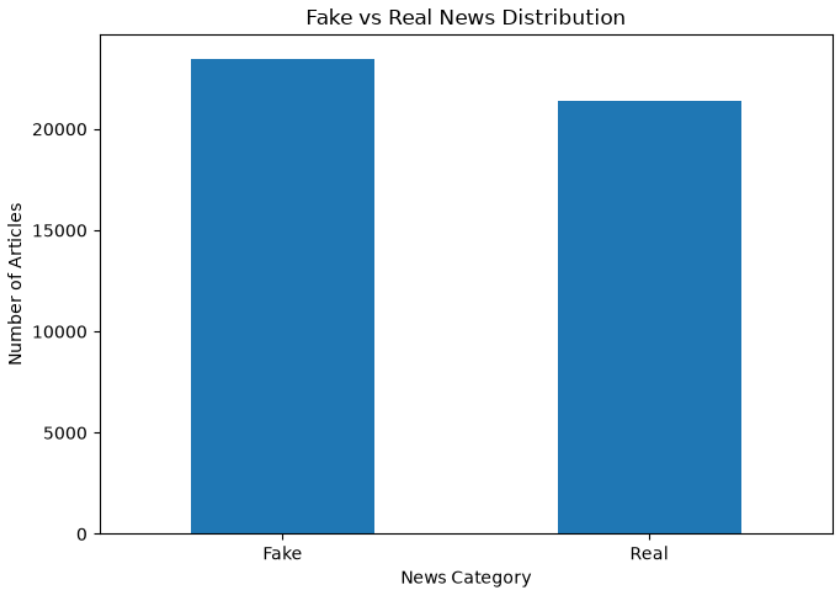


Figure 1. Label distribution in the raw dataset.

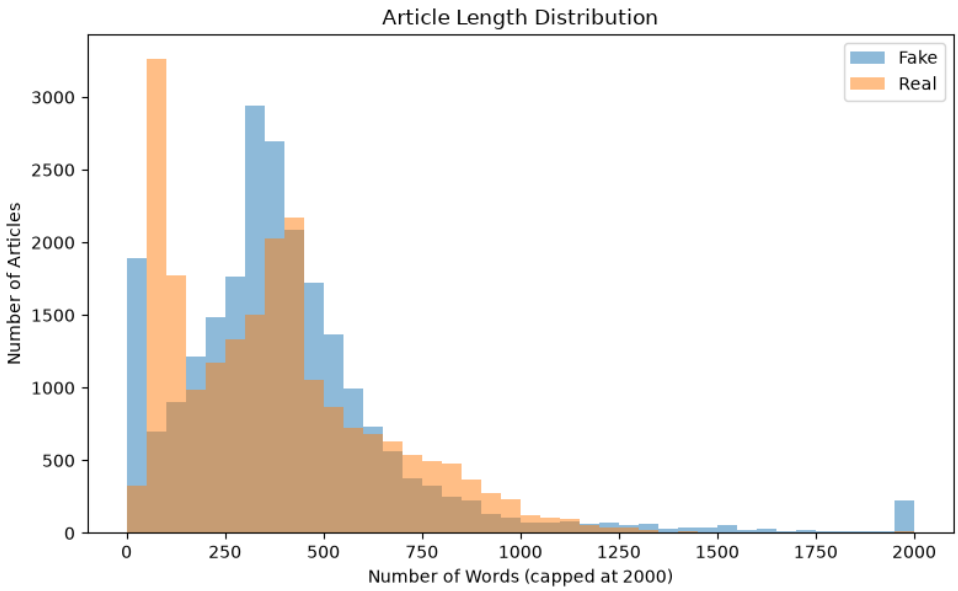


Figure 2. Distribution of article word lengths.

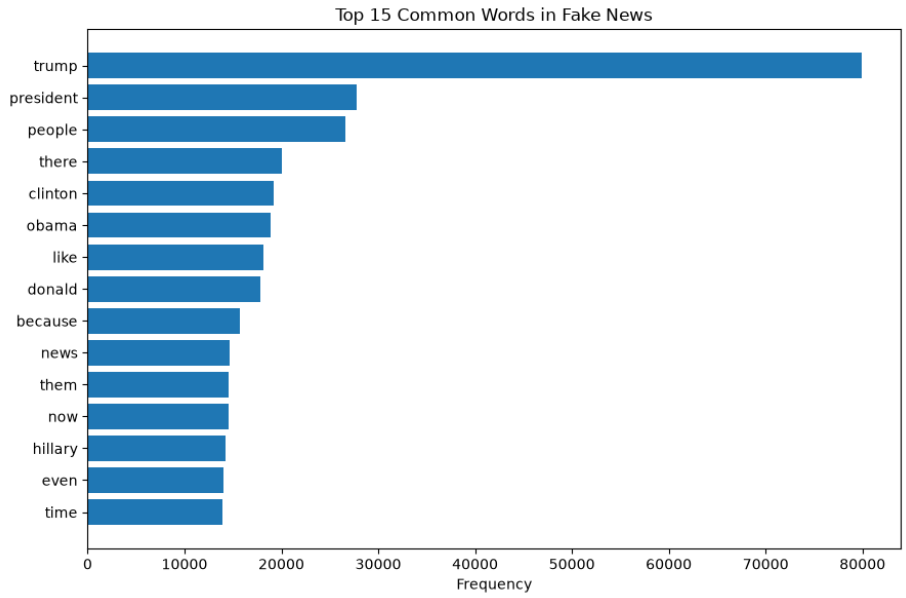


Figure 3. Frequent words in Fake articles.

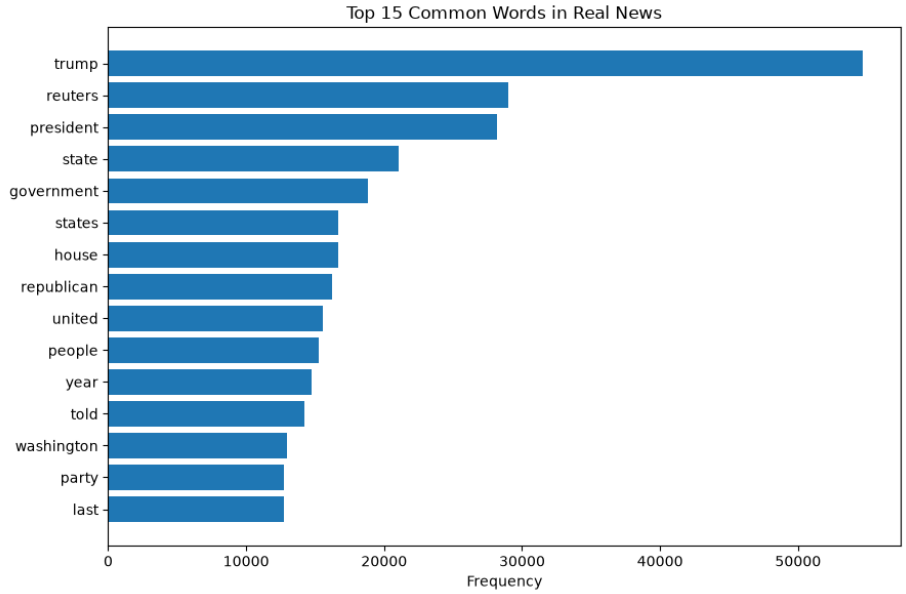


Figure 4. Frequent words in Real articles.

8. Feature Extraction Using TF-IDF

Machine-learning classifiers require numerical input. The project uses TfidfVectorizer to convert cleaned text into a sparse matrix of term weights. TF-IDF increases the importance of terms that are informative within a document while reducing the weight of terms that appear across many documents.

• English stop-word removal.

• Unigrams and bigrams (ngram_range = (1, 2)).

• Maximum vocabulary size: 50,000 features.

• Minimum document frequency: 2; maximum document frequency: 0.9.

• Sublinear term frequency scaling enabled.

The train/test split is performed before fitting TF-IDF. The vectorizer is fitted only on the training text and then used to transform the test set and future user input. Cross-validation uses a pipeline that refits TF-IDF inside each fold, preventing vocabulary/IDF leakage from validation folds.

9. Machine-Learning Algorithms

9.1 Logistic Regression

Logistic Regression is a supervised linear classification algorithm. It calculates a weighted combination of input features and applies a logistic function to estimate the probability of the positive class. In this project it uses C=10 and a maximum of 1,000 iterations. Its learned coefficients also support a simple influential-word explanation.

9.2 Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classifier commonly used for text categorization. It estimates class probabilities from feature statistics and applies a conditional-independence assumption between features. The project uses alpha=0.1 for smoothing. It provides a fast baseline for comparison.

10. Training and Testing Procedure

The cleaned dataset contains 38,824 articles after preprocessing and duplicate removal. A stratified 80:20 train/test split is created using random seed 42. This produces 31,059 training articles and 7,765 test articles. Stratification preserves the class proportions across the split. Both classifiers are fitted on the training features and evaluated on the held-out test features. Five-fold cross-validation is also run on the training set for F1-score stability.

The held-out test set is not used to fit the TF-IDF vocabulary or inverse-document-frequency weights. The vectorizer and model artifacts are saved after training.

11. Performance Metrics and Model Comparison

Accuracy measures the proportion of all predictions that are correct. Precision measures the proportion of predicted Fake articles that are actually Fake. Recall measures the proportion of Fake articles correctly identified. F1-score is the harmonic mean of precision and recall. ROC-AUC measures ranking performance across classification thresholds. Fake is treated as the positive class.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV F1 mean ± std
Logistic Regression	98.98%	99.35%	98.44%	98.89%	99.92%	0.9891 ± 0.0015
Multinomial NB	95.98%	95.69%	95.59%	95.64%	99.16%	0.9563 ± 0.0008

The selected model is **Logistic Regression**. It achieved an F1-score of 98.89% and ROC-AUC of 99.92%, higher than the other evaluated model in this experiment. F1-score is used as the primary selection metric, with ROC-AUC as a secondary comparison.

12. Confusion Matrix and ROC Analysis

A confusion matrix summarizes correct and incorrect predictions for each class. False positives are Real articles incorrectly classified as Fake; false negatives are Fake articles incorrectly classified as Real. Both error types matter: false positives can undermine trust in legitimate reporting, while false negatives can allow misleading content to pass as Real.

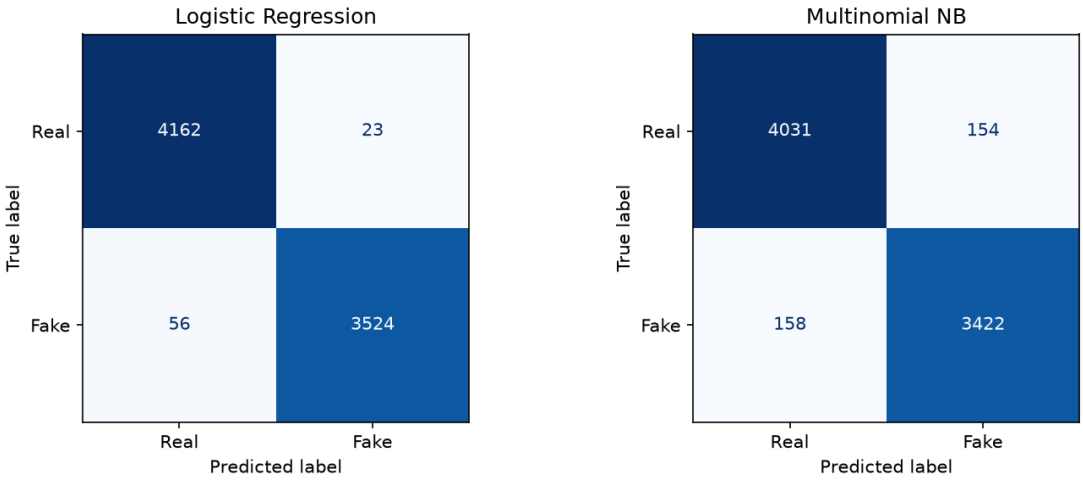


Figure 6. Confusion matrices for the two classifiers.

The ROC curve plots true-positive rate against false-positive rate at different thresholds. The reported ROC-AUC values are 0.9992 for Logistic Regression and 0.9916 for Multinomial Naive Bayes. These values indicate strong separation on this held-out split, but do not guarantee performance on different sources or future events.

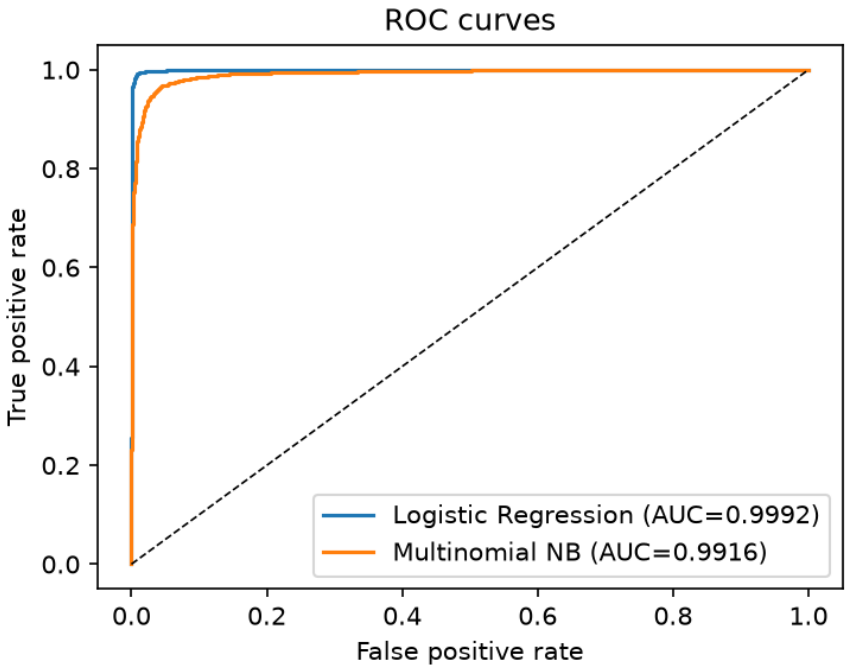


Figure 7. ROC curves for the two classifiers.

13. Streamlit Application Implementation

The Streamlit interface is implemented in app.py. It provides a News Detector tab and a Project Dashboard tab. In the detector, the user can choose a sample example or paste a headline/article body, select Logistic Regression or Multinomial Naive Bayes, and request a prediction. The app validates empty or very short text and checks whether

any input terms are represented in the fitted vocabulary. It displays the predicted class, estimated Fake probability, confidence and influential terms.

The dashboard presents dataset and split counts, TF-IDF vocabulary size, model metrics, EDA charts, confusion matrices and ROC curves. The app also warns that the classifier learns dataset-specific patterns and cannot independently verify facts.

14. Model Persistence Using Joblib

After training, the project saves the fitted TF-IDF vectorizer and both classifiers as Joblib files in the models directory. It also saves best_model.joblib and metrics.json. The Streamlit app loads the saved vectorizer and classifiers when it starts; it does not retrain the models for each prediction. This separates model development from application inference and makes the app faster to launch.

• models/tfidf_vectorizer.joblib

• models/logistic_regression.joblib

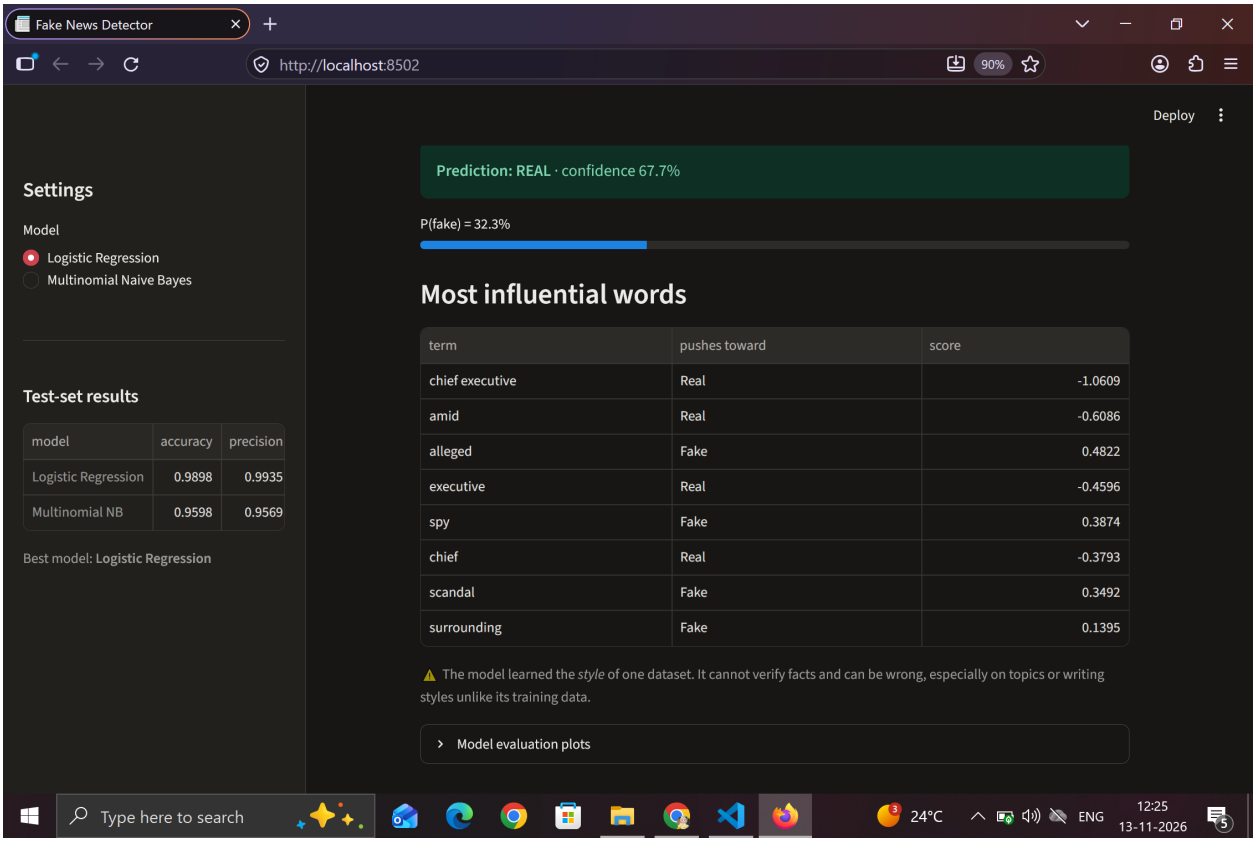
• models/multinomial_nb.joblib

• models/best_model.joblib

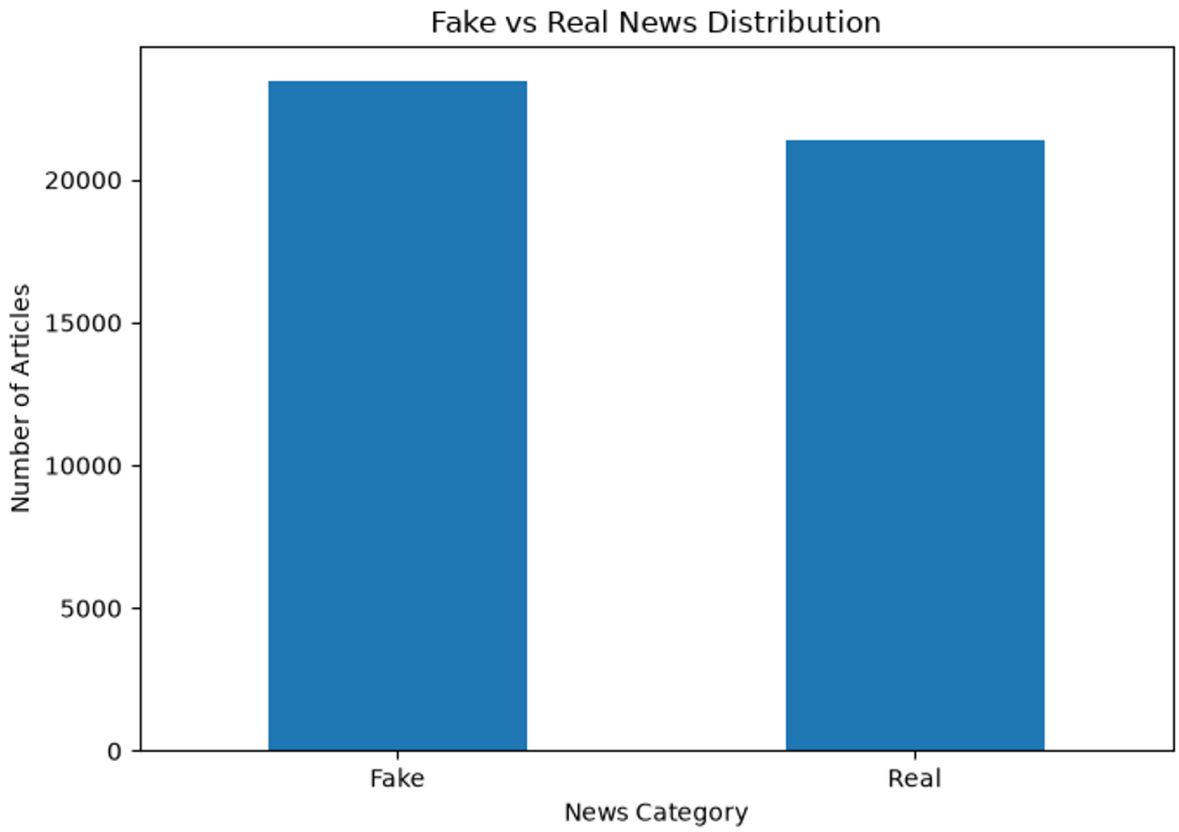
• models/metrics.json

15. Screenshots and Sample Predictions

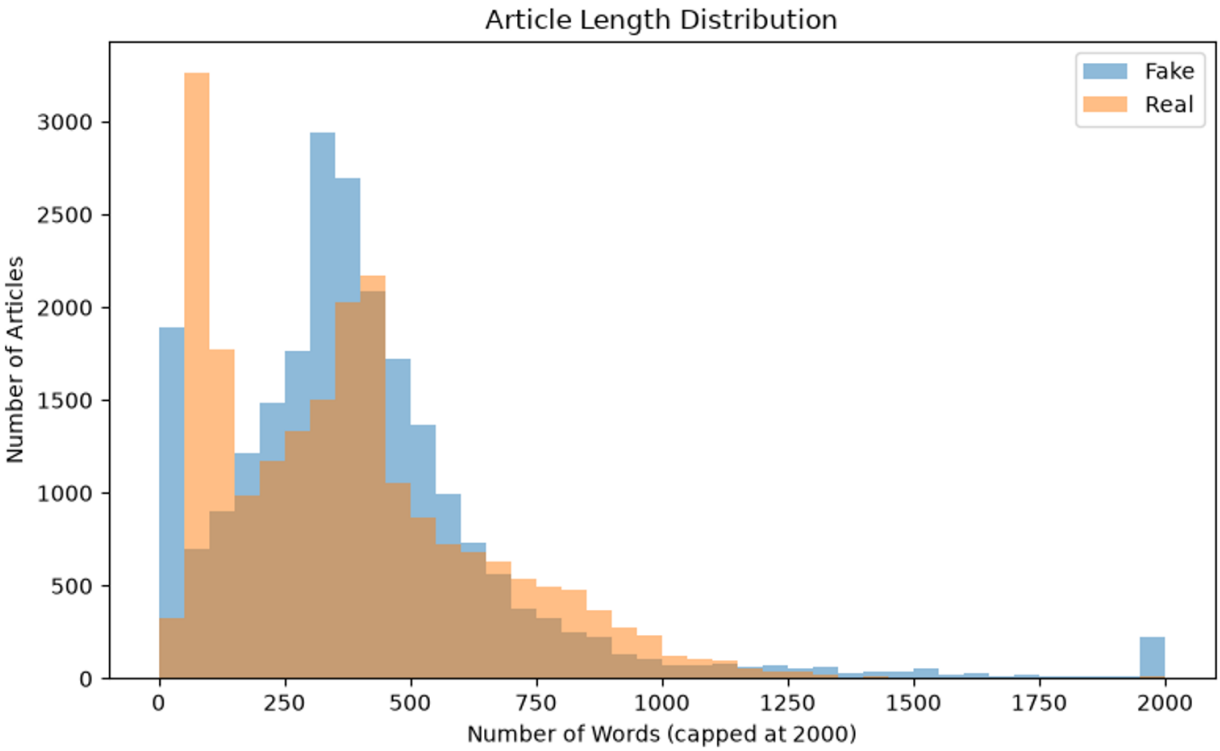
Application screenshots should document the dataset overview, EDA, evaluation metrics, model comparison/confusion matrices, main app interface, a prediction result and validation of empty or invalid input. Place screenshot files in the screenshots directory and rerun this script to embed them below. Screenshots are not fabricated or substituted with mock application output.



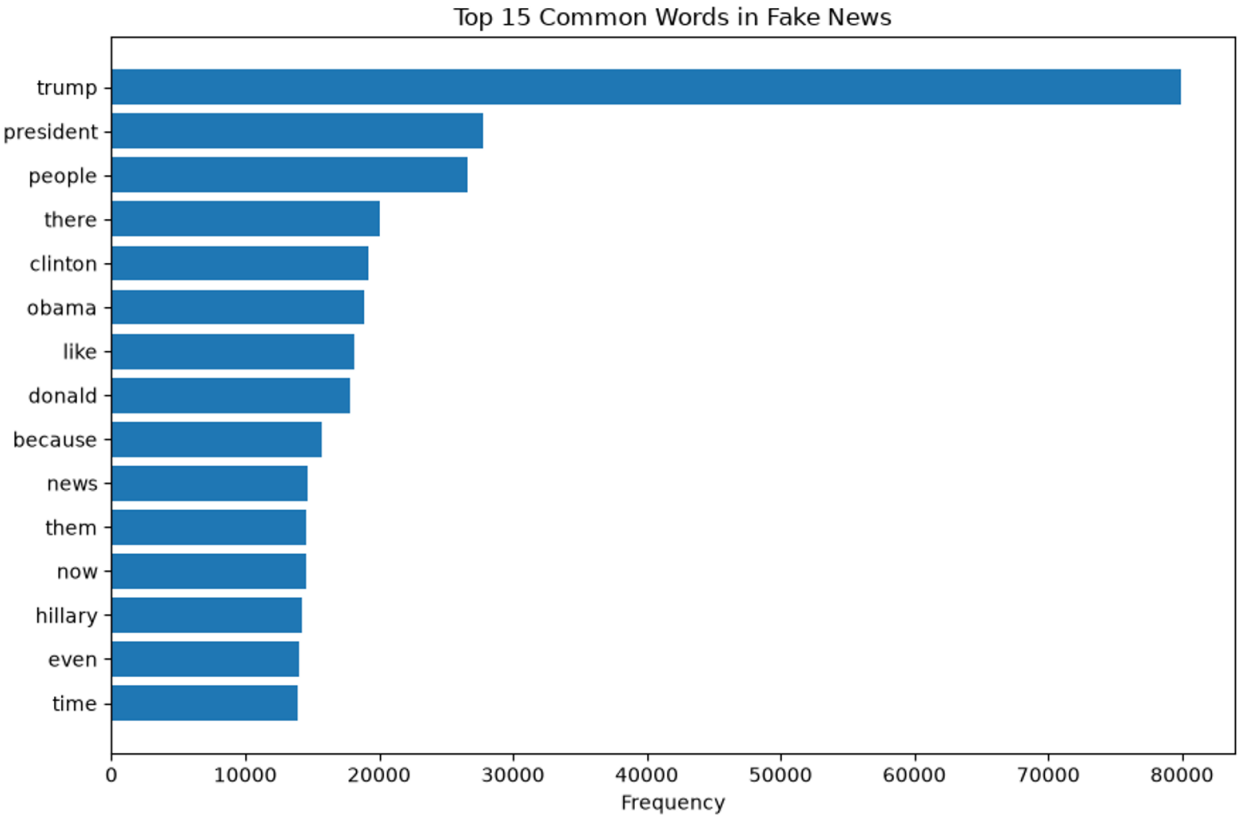
Application screenshot: s10_prediction2.PNG



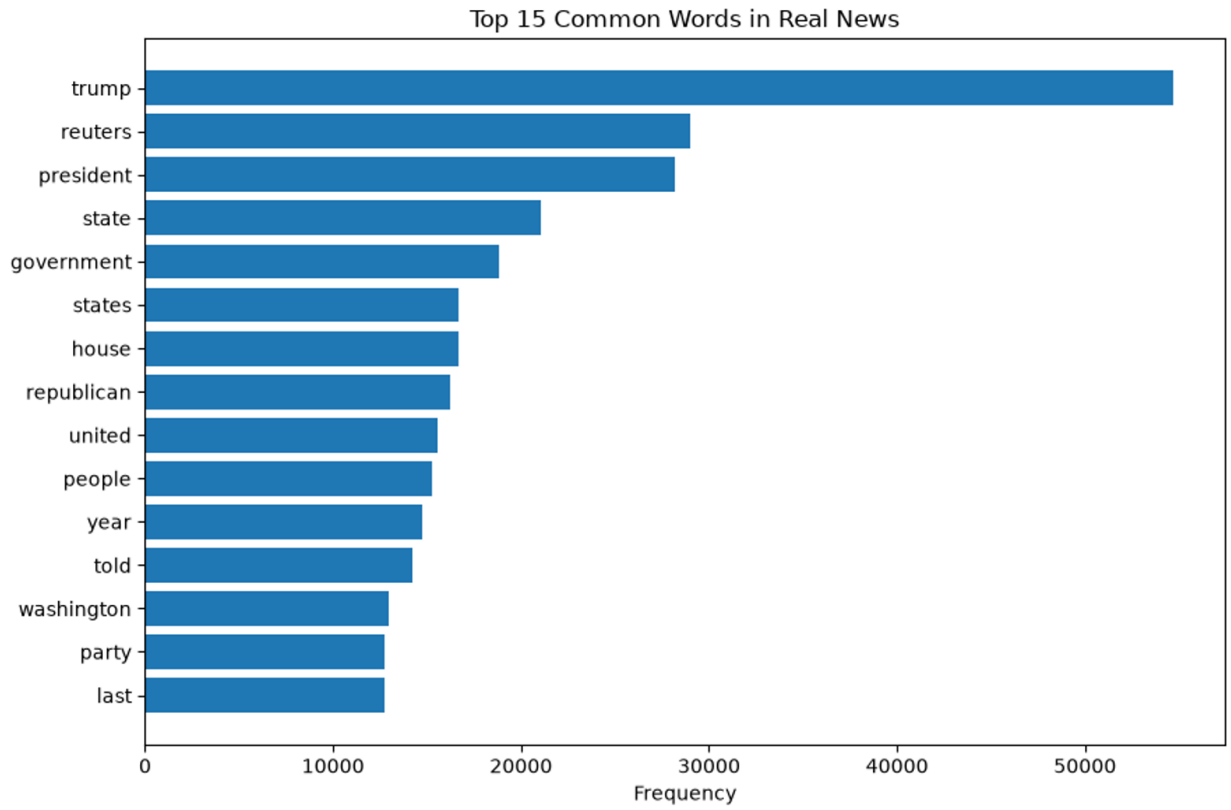
Application screenshot: s1_dataset_overview.png



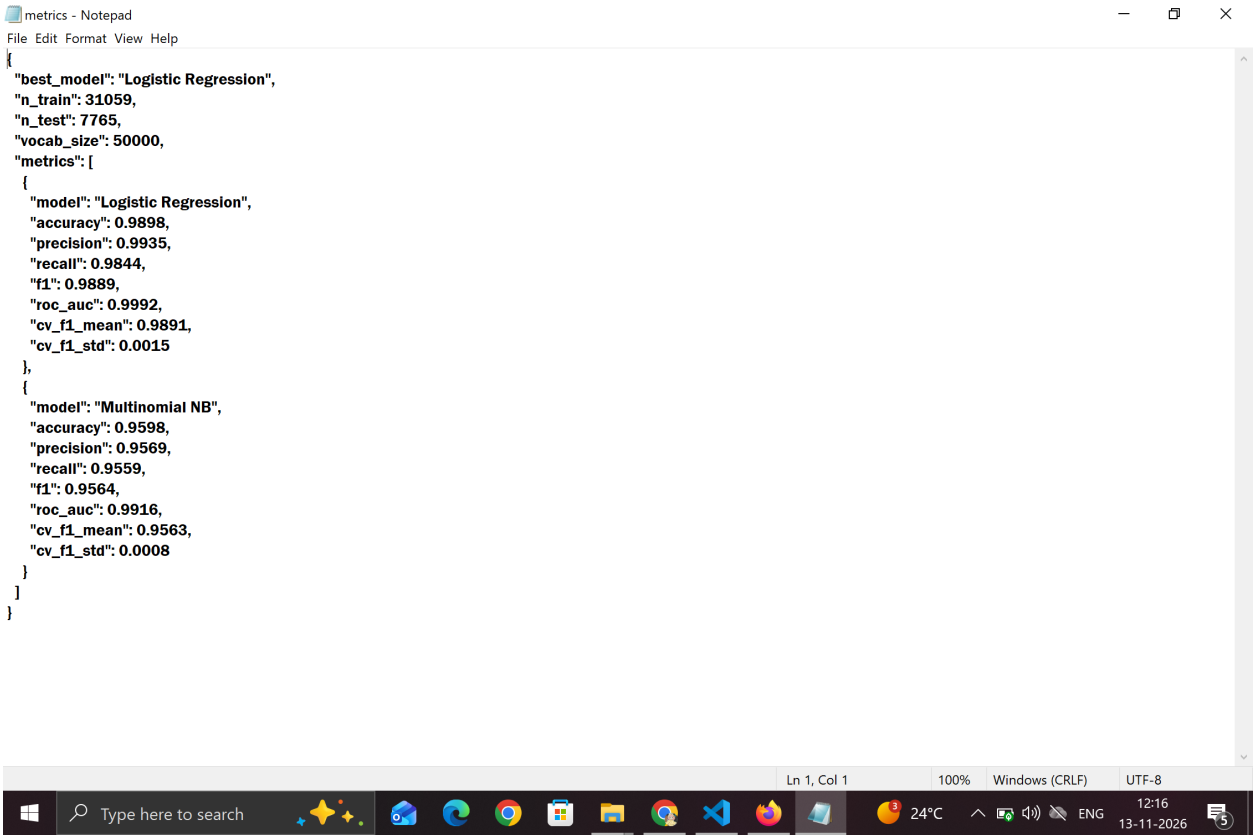
Application screenshot: s2_article_lenght.PNG



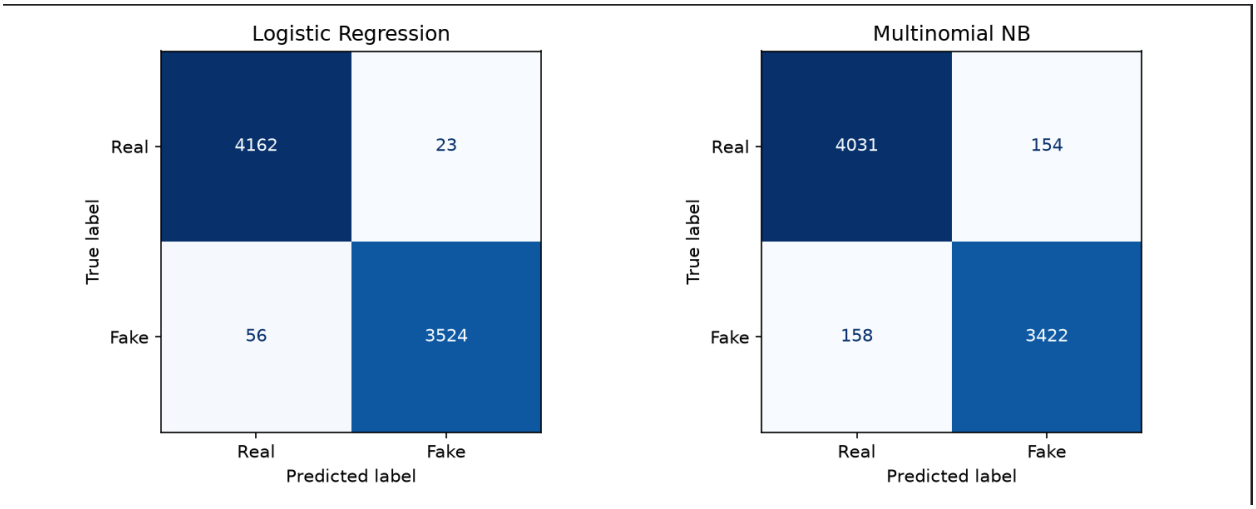
Application screenshot: s3_commonwords_fake.PNG



Application screenshot: s4_commonwords_real.PNG



Application screenshot: s5_model_metrics.png



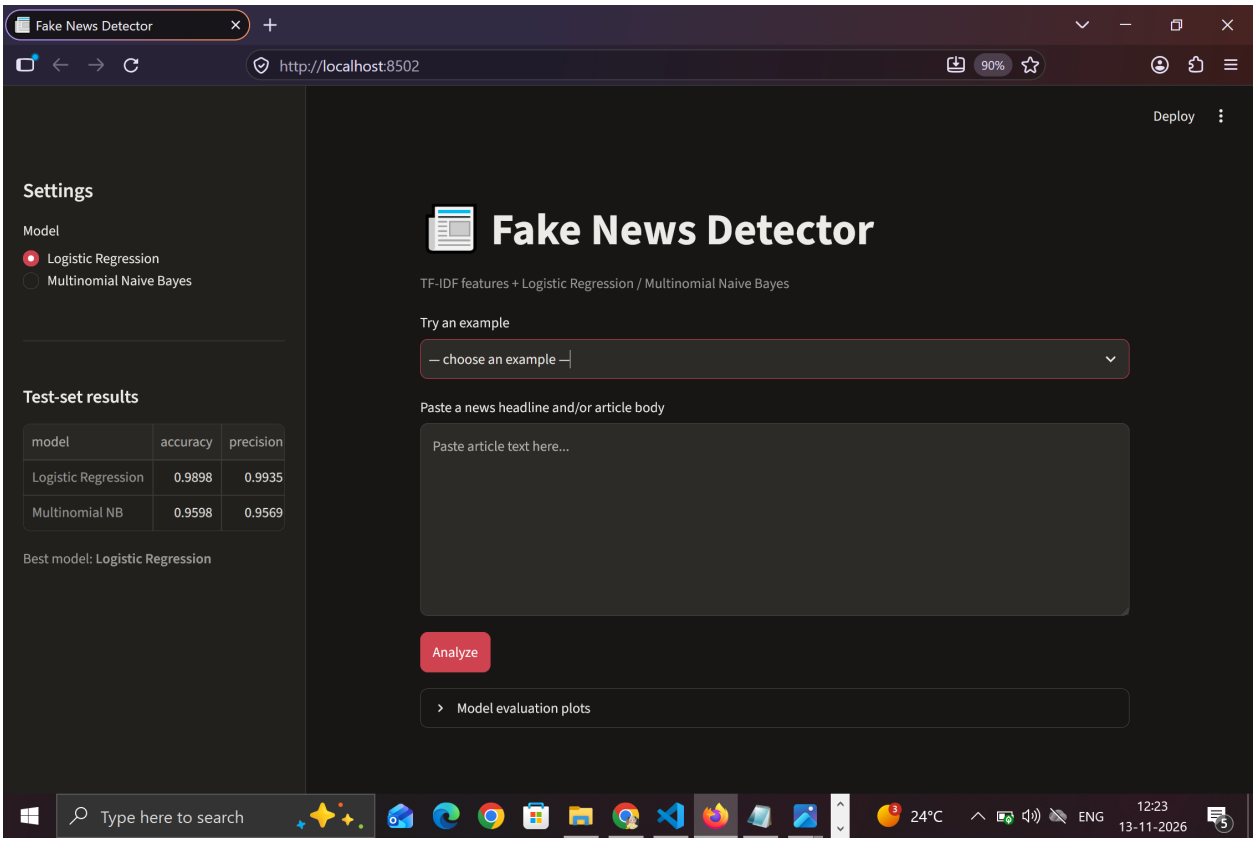
Application screenshot: s6_confusion_matrices.png

Clipboard | Font | Alignment | Number

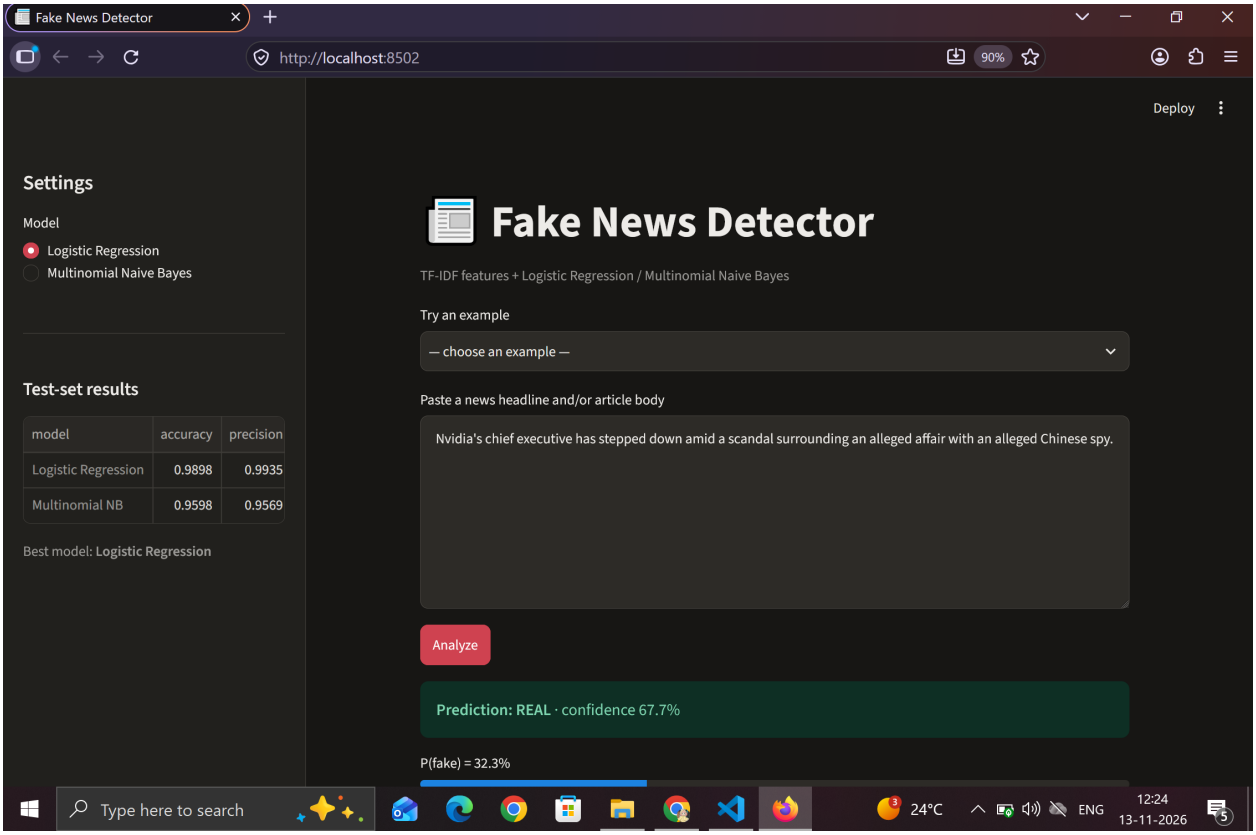
A1 | model

	A	B	C	D	E	F	G	H
1	model	accuracy	precision	recall	f1	roc_auc	cv_f1_mea	cv_f1_std
2	Logistic Reg	0.9898	0.9935	0.9844	0.9889	0.9992	0.9891	0.0015
3	Multinomia	0.9598	0.9569	0.9559	0.9564	0.9916	0.9563	0.0008

Application screenshot: s7_model_comparison.PNG



Application screenshot: s8_interface.PNG



Application screenshot: s9_prediction1.PNG

Sample predictions should be demonstrated using both provided examples and at least one user-entered article. The model output is an estimate based on learned text patterns, not a verified factual verdict.

16. Challenges and Solutions

• **Text is not directly usable by classic classifiers:** addressed by cleaning text and converting it to TF-IDF features.

• **Potential data leakage:** addressed by splitting before vectorizer fitting and refitting the vectorizer inside cross-validation folds.

• **Duplicate articles can inflate evaluation scores:** exact duplicate cleaned texts are removed before splitting.

• **Application should not retrain at every launch:** fitted artifacts are saved with Joblib and loaded by Streamlit.

• **Input can be empty or outside the learned vocabulary:** the interface validates text and warns when no vocabulary terms are recognized.

17. Limitations and Future Work

• The dataset is mainly English-language and includes many political and world-news articles from a particular collection period.

• The model may learn publisher, topic or writing-style signals rather than factual correctness.

• Satire, breaking news, emerging topics, different languages and unfamiliar writing styles may be misclassified.

• The application does not retrieve sources, check citations or independently verify claims.

• A high score on this dataset does not guarantee generalization to current real-world news.

Future work: evaluate on independent and more recent datasets; investigate source and topic leakage; tune hyperparameters; compare with Linear SVM and transformer-based models; calibrate probabilities; add explainability methods; and deploy with monitoring and clear user-facing safeguards.

18. Conclusion and References

The project implements an end-to-end NLP workflow for classifying news text using TF-IDF, Logistic Regression and Multinomial Naive Bayes. The models were compared using a held-out test set and cross-validation. Logistic Regression achieved the strongest results in this experiment and was selected for the project. A Streamlit application loads saved artifacts and presents predictions alongside evaluation and EDA visualizations. The system is a learning and pattern-classification demonstration; it should support human review rather than replace fact-checking.

References

1. ISOT Research Lab, University of Victoria, “Fake News Detection Datasets.” https://onlineacademiccommunity.uvic.ca/isot/2022/11/27/fake-news-detection-datasets/

2. ISOT Research Lab, University of Victoria, “ISOT Fake News Dataset ReadMe.” https://onlineacademiccommunity.uvic.ca/isot/wp-content/uploads/sites/7295/2023/02/ISOT_Fake_News_Dataset_ReadMe.pdf

3. Ahmed, H., Traore, I., and Saad, S. (2018). “Detecting opinion spams and fake news using text classification.” Journal of Security and Privacy, 1, e9.

4. Scikit-learn documentation: TF-IDF Vectorizer, Logistic Regression, Multinomial Naive Bayes and model evaluation metrics. https://scikit-learn.org/stable/

5. Streamlit documentation. https://docs.streamlit.io/

6. Joblib documentation. https://joblib.readthedocs.io/